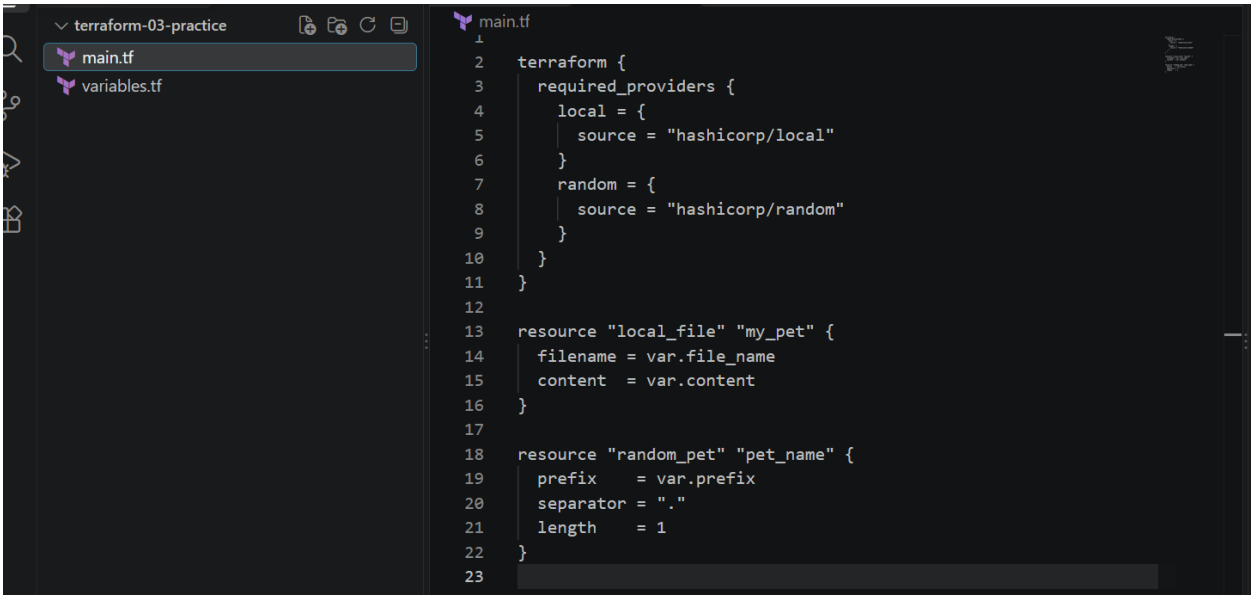
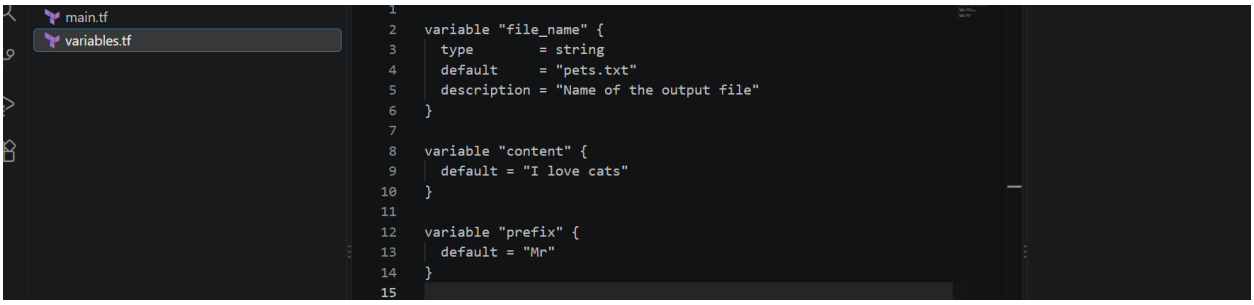
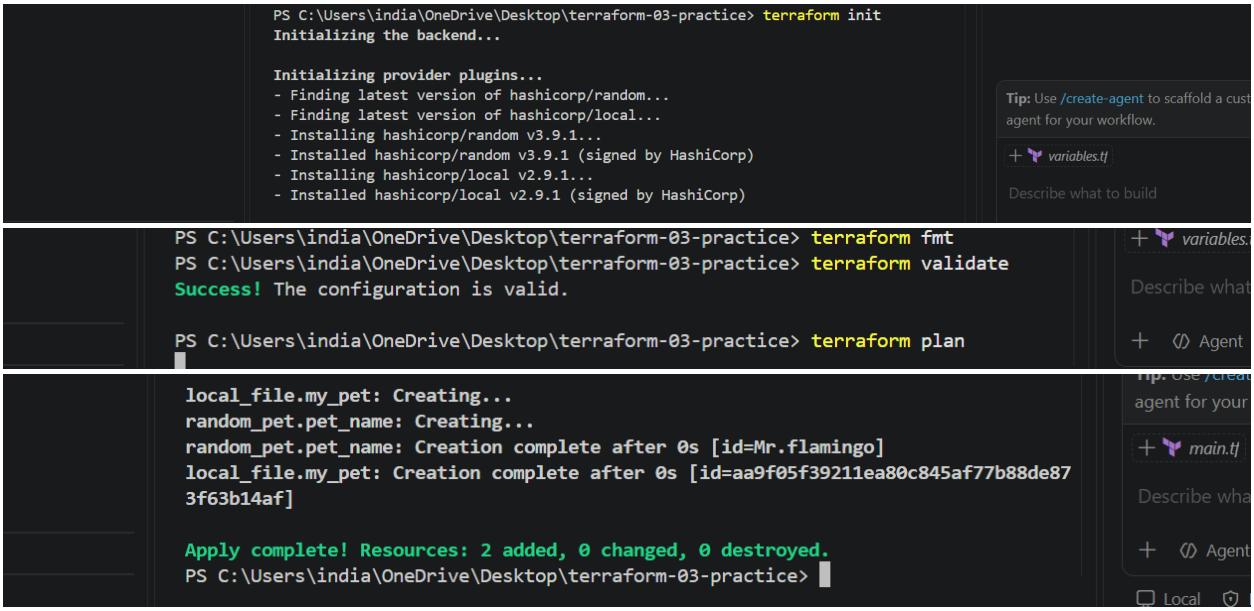
Terraform 3

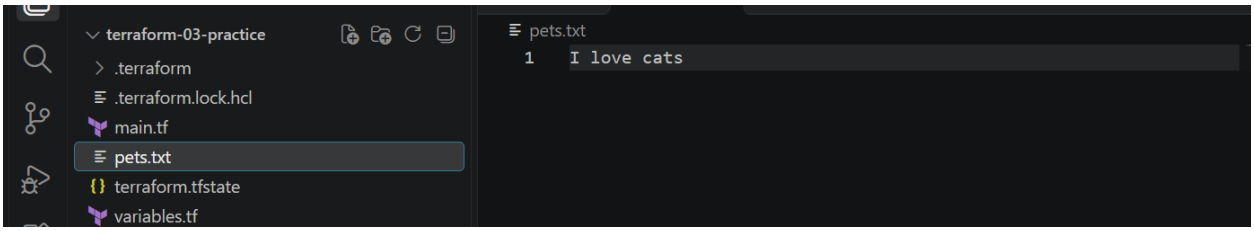


Create main.tf and add the script



Also the variable.tf





This displays details of the resources managed by Terraform.



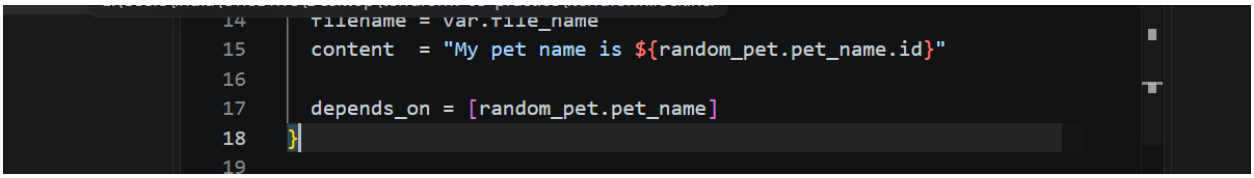
Replace the main file



Now the content changed

Terraform automatically understands that the local file depends on the random pet resource because we referenced its ID. This is called an implicit dependency.

Depends on:



Your configuration already has an implicit dependency through `${random_pet.pet_name.id}`. Adding `depends_on` here is for learning the syntax; it is redundant in this particular example.



Add this at the bottom of the main file

Your output block reads the ID generated by the Random provider.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "Mr.flamingo"
PS C:\Users\india\OneDrive\Desktop\terraform-03-practice>
```

`output` displays resource information.

`random_pet.pet_name.id` retrieves the generated pet name.

`terraform output` displays saved output values without applying the configuration again.

```
PS C:\Users\india\OneDrive\Desktop\terraform-03-practice> terraform plan -var="pr
efix=King"
random_pet.pet_name: Refreshing state... [id=Mr.flamingo]
local_file.my_pet: Refreshing state... [id=41b00f9fbb79783754c8fc45d6df0640b3cfed
aa1]
```

```
  # random_pet.pet_name must be replaced
-/+ resource "random_pet" "pet_name" {
      ~ id        = "Mr.flamingo" -> (known after apply)
      ~ prefix    = "Mr" -> "King" # forces replacement
        # (2 unchanged attributes hidden)
    }
```

This will actually create the resources using the `King` prefix

Environment variables:

```
  # random_pet.pet_name must be replaced
-/+ resource "random_pet" "pet_name" {
      ~ id        = "Mr.flamingo" -> (known after apply)
      ~ prefix    = "Mr" -> "Tiger" # forces replacement
        # (2 unchanged attributes hidden)
    }
```

Terraform automatically reads environment variables with the `TF_VAR_` prefix

So `TF_VAR_prefix` overrides the default `prefix = "Mr"` with `Tiger`.

Interactive variable input:

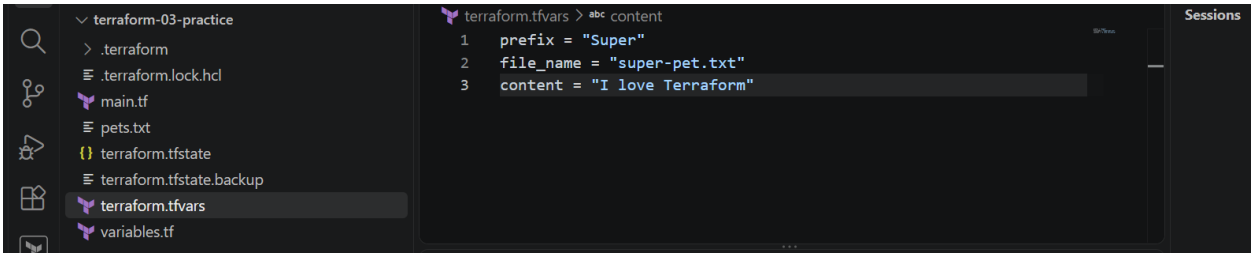
```
PS C:\Users\india\OneDrive\Desktop\terraform-03-practice> Remove-Item Env:TF_VAR_
prefix -ErrorAction SilentlyContinue
>> terraform plan
var.prefix
  Enter a value: Lion
```

```
  # random_pet.pet_name must be replaced
-/+ resource "random_pet" "pet_name" {
      ~ id        = "Mr.flamingo" -> (known after apply)
      ~ prefix    = "Mr" -> "Lion" # forces replacement
        # (2 unchanged attributes hidden)
    }

Plan: 2 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  ~ pet_name = "Mr.flamingo" -> (known after apply)
```

Variable files (`terraform.tfvars`)

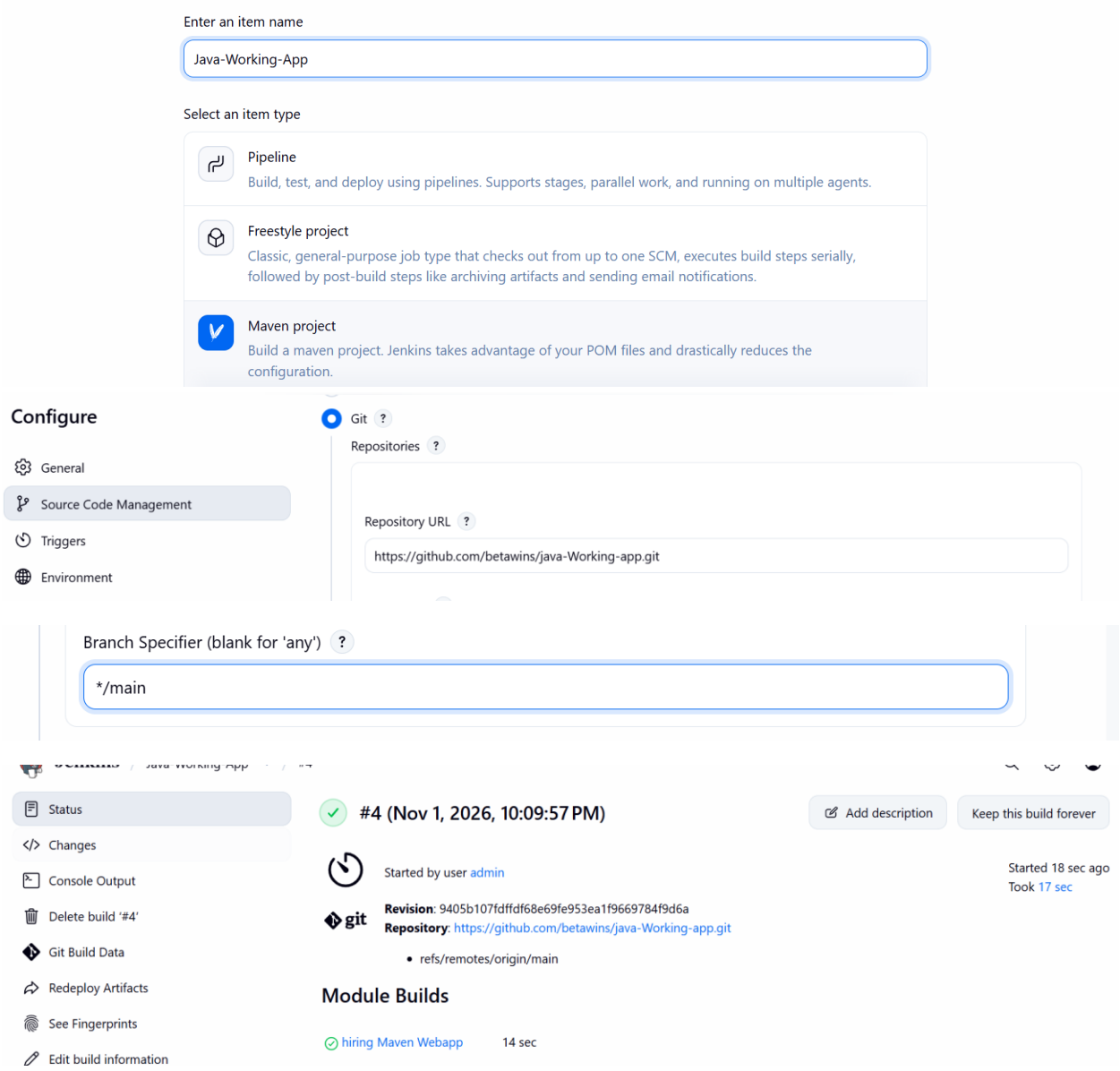


Terraform automatically loads `terraform.tfvars` when you run `terraform plan` or `terraform apply`.

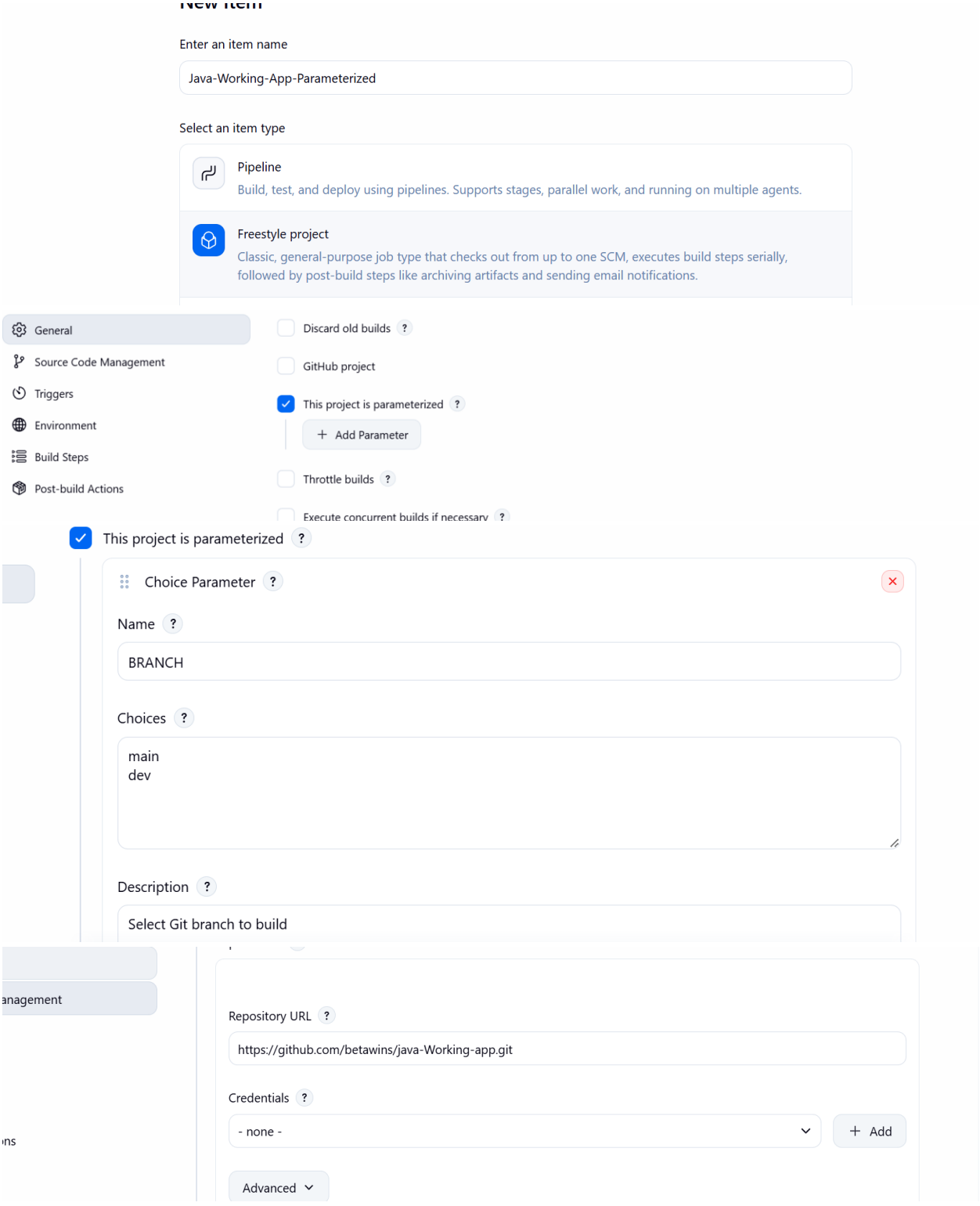
```
  # random_pet.pet_name must be replaced
-/+ resource "random_pet" "pet_name" {
      ~ id        = "Mr.flamingo" -> (known after apply)
      ~ prefix    = "Mr" -> "Super" # forces replacement
        # (2 unchanged attributes hidden)
    }
```

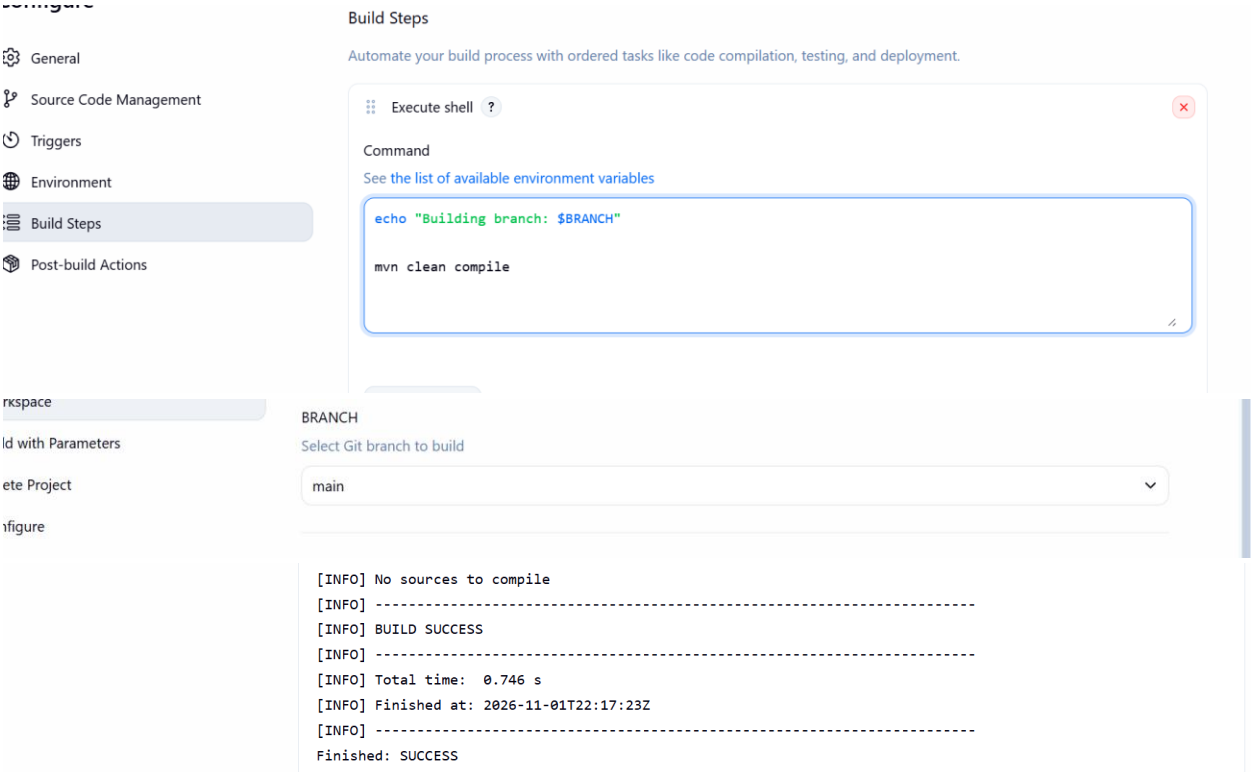
Creating a Maven Project in Jenkins:

Install the maven integration plugin



Create a Parameterized Jenkins Job


Enter an item name
Java-Working-App-Parameterized
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Discard old builds
GitHub project
This project is parameterized
Add Parameter
Throttle builds
This project is parameterized
Choice Parameter
Name
BRANCH
Choices
main
dev
Description
Select Git branch to build
Repository URL
https://github.com/betawins/java-Working-app.git
Credentials
- none -
Add
Advanced



What are Jenkins Global Variables?

Jenkins provides predefined environment variables that contain information about the current job, build, workspace, and Jenkins server.

We can use these variables in shell scripts and Jenkins pipelines.

Example: Instead of manually entering a build number, Jenkins automatically provides it through `$BUILD_NUMBER`.

2. Important Jenkins Global Variables

Variable	Description
`BUILD_NUMBER`	Current build number
`BUILD_ID`	Unique build identifier
`BUILD_URL`	URL of the current build

JOB_NAME	**Name of the Jenkins job**
JOB_BASE_NAME	**Short job name**
WORKSPACE	**Directory where the job runs**
JENKINS_HOME	**Jenkins home directory**
JENKINS_URL	**Jenkins server URL**
NODE_NAME	**Name of the node running the build**
EXECUTOR_NUMBER	**Executor number running the build**
BUILD_TAG	**Unique identifier for the build**
GIT_BRANCH	**Git branch, when provided by Git integration**
GIT_COMMIT	**Git commit hash, when provided by Git integration**